CartPole Stabilization - Proximal Policy Optimization

Osman Melih Muslu

08.10.26

1 Problem Formulation

State vector

$$s_t = \begin{bmatrix} x_t & \dot{x}_t & \vartheta_t & \dot{\vartheta}_t \end{bmatrix}^T \tag{1}$$

şeklinde tanımlanır.

İstenen equilibrium point

$$s_{\text{ref}} = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T \tag{2}$$

olarak tanımlanmıştır.

Agent, scalar ve continuous bir control action üretir:

$$a_t \in \mathbb{R}. \tag{3}$$

Action fiziksel yatay kuvvete

$$u_{t,\text{cmd}} = 10a_t \tag{4}$$

şeklinde physical force command'a dönüştürülür. Environment tarafından uygulanacak gerçek control input

$$u_t = \text{clip}\left(u_{t,\text{cmd}}, -10, 10\right) \tag{5}$$

şeklinde sınırlandırılır.

Control input

$$-10 \text{ N} \leq u_t \leq 10 \text{ N} \tag{6}$$

aralığında sınırlandırılmıştır.

2 Reinforcement Learning Structure

Kullanılan PPO controller; *model-free*, *on-policy*, *actor–critic* ve *continuous-action* bir reinforcement learning algoritmasıdır.

Yöntem *model-free*'dir, learning environment ile olan interactionla olur:

$$(s_t, a_t, r_{t+1}, s_{t+1}). \tag{7}$$

Actor–critic: Actor, control policy'yi temsil eder:

$$\pi_\theta(a|s), \tag{8}$$

burada θ actor parameters'dır.

Critic ise state-value function'ı yaklaşık olarak temsil eder:

$$V_\phi(s) \approx V^\pi(s), \tag{9}$$

burada ϕ critic parameters'dır.

3 Reinforcement Learning Objective

Expected discounted return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[G_t\right]. \tag{10}$$

Discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{11}$$

şeklinde tanımlanır.

Buradaki discount factor γ, future rewards'un current decision üzerindeki önemini belirler. γ değerinin 1'e yaklaşması long-term rewards'a daha fazla önem verilmesini sağlarken, daha düşük γ değerleri daha short-term bir optimization davranışına yol açar.

Baseline configuration için

$$\gamma = 0.98 \tag{12}$$

kullanılmaktadır.

4 Reward Function

Reinforcement learning objective, quadratic state ve control cost kullanılarak tanımlanmaktadır:

$$\begin{aligned} \mathcal{J}_t = q_x(x_t - x_{\text{ref}})^2 + q_{\dot{x}}\dot{x}_t^2 + q_\vartheta \vartheta_t^2 \\ + q_{\dot{\vartheta}}\dot{\vartheta}_t^2 + R_u u_{t,\text{cmd}}^2 \end{aligned} \tag{13}$$

Baseline configuration için

$$q_x = q_{\dot{x}} = q_\vartheta = q_{\dot{\vartheta}} = 1, \qquad R_u = 0.1 \tag{14}$$

olarak seçilmiştir.

Instantaneous reward, cost'un pozitif ve bounded bir reward'a dönüştürülmesiyle elde edilir:

$$r_t = \exp(-\mathcal{J}_t). \tag{15}$$

Dolayısıyla

$$0 < r_t \leq 1. \tag{16}$$

5 Actor–Critic Neural Network Architecture

PPO controller, policy ve value function'ı approximate etmek için iki ayrı neural network kullanmaktadır: an *actor* network ve a *critic* network.

CartPole state vector

$$\mathbf{s}_t = \begin{bmatrix} x_t & \dot{x}_t & \vartheta_t & \dot{\vartheta}_t \end{bmatrix}^T \tag{17}$$

olduğundan, her iki network de dört-dimensional state vector'ı input olarak almaktadır.

5.1 Actor Network

Actor, stochastic policy'yi temsil etmektedir:

$$\pi_\theta(a|\mathbf{s}). \tag{18}$$

Kullanılan implementation'da actor, iki hidden layer içeren bir multilayer perceptron (MLP) olarak tanımlanmıştır. Network architecture

$$\boxed{4 \to 32 \to 32 \to 1} \tag{19}$$

şeklindedir.

Hidden layers içerisinde LeakyReLU activation function kullanılmaktadır. Actor network'ün output'u continuous Gaussian policy'nin mean değerini belirler:

$$\mu_\theta(\mathbf{s}_t). \tag{20}$$

Continuous action distribution

$$\pi_\theta(a|\mathbf{s}_t) = \mathcal{N}\left(\mu_\theta(\mathbf{s}_t), \sigma^2\right) \tag{21}$$

şeklinde ifade edilir.

Burada $\mu_\theta(\mathbf{s}_t)$ state-dependent olarak actor neural network tarafından hesaplanırken, standard deviation σ doğrudan network output'u değildir. Implementation içerisinde $\log \sigma$ ayrı bir trainable parameter olarak tutulmaktadır:

$$\sigma = \exp(\log \sigma). \tag{22}$$

Training sırasında control action Gaussian distribution'dan sample edilir:

$$a_t \sim \mathcal{N}\left(\mu_\theta(\mathbf{s}_t), \sigma^2\right). \tag{23}$$

Bu stochastic sampling exploration sağlar. Deterministic evaluation sırasında ise distribution'dan random sampling yapmak yerine actor mean action kullanılır:

$$a_t = \mu_\theta(\mathbf{s}_t). \tag{24}$$

5.2 Critic Network

Critic network, current policy altında state-value function'ı approximate etmektedir:

$$V_\phi(\mathbf{s}) \approx V^\pi(\mathbf{s}). \tag{25}$$

Critic de actor ile aynı hidden-layer structure'a sahip ayrı bir MLP'dir:

$$\boxed{4 \to 32 \to 32 \to 1} \tag{26}$$

Critic'in scalar output'u

$$V_\phi(\mathbf{s}_t) \tag{27}$$

current state'ten başlanıp current policy takip edildiğinde elde edilmesi beklenen discounted future return'ü tahmin etmektedir.

Dolayısıyla actor ve critic aynı CartPole state vector'ını kullanmasına rağmen farklı amaçlara sahiptir:

$$\boxed{\begin{array}{ll} \text{Actor:} & \mathbf{s}_t \to \mu_\theta(\mathbf{s}_t) \to \pi_\theta(a|\mathbf{s}_t), \\ \text{Critic:} & \mathbf{s}_t \to V_\phi(\mathbf{s}_t). \end{array}} \tag{28}$$

Actor parameters θ PPO clipped policy objective kullanılarak, critic parameters ϕ ise value-function loss kullanılarak ayrı ayrı optimize edilmektedir.

6 Policy Gradient Principle

Stochastic policy kullanan bir reinforcement learning agent için temel amaç, policy parameters θ'yı, expected return'ü artıracak yönde güncellemektir. Bu amaç

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_t] \tag{29}$$

ile ifade edilir. Burada $J(\theta)$ mevcut policy'nin expected performance'ını, G_t ise gelecekte elde edilecek discounted return'ü göstermektedir.

Dolayısıyla training sırasında temel soru şudur:

$$\text{“Policy parameters } \theta \text{ hangi yönde değiştirilirse } J(\theta) \text{ artar?”} \tag{30}$$

Bu yön, objective function'ın policy parameters'a göre gradient'i olan

$$\nabla_\theta J(\theta) \tag{31}$$

ile belirlenir.

Parameterized stochastic policy için Policy Gradient Theorem bu gradient'i

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) Q^\pi(s_t, a_t) \right] \tag{32}$$

şeklinde ifade eder.

Bu denklem iki temel parçanın birlikte kullanılmasına dayanır. İlk terim

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \tag{33}$$

seçilen a_t action'ının current state s_t altında daha olası hale gelmesi için policy parameters'ın hangi yönde değiştirilmesi gerektiğini belirler.

Bunun nedeni gradient'in, bir function'ı en hızlı artıran yönü göstermesidir. Burada artırılmak istenen function

$$\log \pi_\theta(a_t|s_t) \tag{34}$$

olduğundan, parameters'ın

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \tag{35}$$

yönünde değiştirilmesi, selected action'ın policy altındaki probability density'sini artırır.

Başka bir ifadeyle,

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \tag{36}$$

terimi şu soruya cevap verir:

"Bu action'ı daha olası yapmak istersem policy parameters'ı hangi yönde değiştirmeliyim?" (37)

Ancak bu terim tek başına selected action'ın iyi veya kötü olduğunu söylemez. Action'ın long-term olarak ne kadar başarılı olduğunu değerlendiren ikinci terim

$$Q^\pi(s_t, a_t) \tag{38}$$

olup, action-value function olarak adlandırılır.

Action-value function

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi \left[G_t \mid S_t = s_t, A_t = a_t \right] \tag{39}$$

şeklinde tanımlanır.

Bu ifade, agent'ın s_t state'inde a_t action'ını uygulaması ve daha sonra current policy π'yi takip etmesi durumunda beklenen future discounted return'ü temsil eder.

Dolayısıyla $Q^\pi(s_t, a_t)$ sadece action'ın anlık reward'unu değil, action'ın future states ve future rewards üzerindeki uzun dönem etkisini de dikkate alır.

CartPole açısından düşünüldüğünde, aynı state için bir action pole'u equilibrium point'e yaklaştırıyor, cart position'ı iyileştiriyor ve sonraki timesteps boyunca yüksek reward alınmasını sağlıyorsa, bu action için

$$Q^\pi(s_t, a_t) \tag{40}$$

yüksek olacaktır.

Buna karşılık action pole'u equilibrium point'ten uzaklaştırıyor veya future rewards'un azalmasına neden oluyorsa, ilgili action-value daha düşük olacaktır.

Dolayısıyla policy-gradient learning'in temel fikri, iyi long-term sonuçlar üreten action'ların policy altında daha olası hale gelmesini sağlamaktır.

Actor–critic algoritmalarında doğrudan $Q^\pi(s_t, a_t)$ kullanmak yerine çoğunlukla advantage function kullanılır:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t). \tag{41}$$

Bu durumda policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) A^\pi(s_t, a_t)\right] \tag{42}$$

şeklinde yazılabilir.

Bu ifade öğrenme mekanizmasını daha açık hale getirir. Eğer

$$A^\pi(s_t, a_t) > 0, \tag{43}$$

ise selected action mevcut state için beklenenden daha iyi performans göstermiştir ve policy update bu action'ın probability'sini artıracak yönde gerçekleştirilir.

Eğer

$$A^\pi(s_t, a_t) < 0, \tag{44}$$

ise selected action beklenenden daha kötü sonuç üretmiştir. Negative advantage, policy gradient yönünü tersine çevirerek bu action'ın probability'sinin azaltılmasını sağlar.

Bu nedenle policy-gradient learning'in temel mantığı

$$\boxed{\begin{array}{lcl} A_t > 0 & \Rightarrow & \pi_\theta(a_t|s_t) \text{ artırılır,} \\ A_t < 0 & \Rightarrow & \pi_\theta(a_t|s_t) \text{ azaltılır.} \end{array}} \tag{45}$$

şeklinde özetlenebilir.

Dolayısıyla

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \tag{46}$$

policy parameters'ın değiştirilmesi gereken yönü belirlerken,

$$A^\pi(s_t, a_t) \tag{47}$$

bu yönde mi yoksa ters yönde mi update yapılması gerektiğini ve update'in göreli önemini belirler.

7 Critic and State-Value Function

Actor–critic yapısında critic, state-value function'ı yaklaşık olarak öğrenir.

State-value function

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right] \tag{48}$$

şeklinde tanımlanır.

Critic tarafından

$$V_\phi(s) \approx V^\pi(s) \tag{49}$$

şeklinde approximate edilir.

Dolayısıyla critic'in görevi, mevcut state'ten başlanıp current policy takip edildiğinde elde edilmesi beklenen future discounted return'ü tahmin etmektir.

8 Return and Advantage Estimation

Return recursive olarak

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \cdots \tag{50}$$

$$G_t = r_{t+1} + \gamma G_{t+1} \tag{51}$$

şeklinde hesaplanır.

Rollout gerçek terminal state ile bitmiyorsa, rollout boundary sonrasındaki future return critic value kullanılarak bootstrap edilebilir.

Baseline advantage estimate

$$\hat{A}_t = G_t - V_\phi(s_t) \tag{52}$$

şeklindedir.

Bu denklem, rollout sonucunda elde edilen return ile critic'in önceden beklediği return arasındaki farkı ifade etmektedir.

Eğer

$$G_t > V_\phi(s_t) \tag{53}$$

ise

$$\hat{A}_t > 0 \tag{54}$$

olur.

Bu durumda action sonrasında gerçekleşen long-term sonuç, critic'in beklentisinden daha iyidir.

Buna karşılık

$$G_t < V_\phi(s_t) \tag{55}$$

durumunda

$$\hat{A}_t < 0 \tag{56}$$

elde edilir.

Practical implementation içerisinde rollout batch üzerinden elde edilen advantage estimates optimization öncesinde normalize edilmektedir:

$$\tilde{A}_t = \frac{\hat{A}_t - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-6}}. \tag{57}$$

Burada $\mu_{\hat{A}}$ ve $\sigma_{\hat{A}}$, rollout batch içerisindeki advantage estimates'in mean ve standard deviation değerleridir. Normalize edilmiş $\tilde{A}_t$ PPO actor objective içerisinde kullanılmaktadır.

9 Old and New Policy Probability Ratio

Rollout tamamlandıktan sonra, rollout sırasında sample edilen aynı state–action pairs new policy altında tekrar değerlendirilir.

PPO probability ratio'yu

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{58}$$

şeklinde tanımlar.

Implementation içerisinde rollout sırasında selected action'a ait old-policy log-probability

$$\log \pi_{\theta_{\text{old}}}(a_t|s_t) \tag{59}$$

buffer içerisinde saklanmaktadır. Optimization sırasında current policy aynı state–action pair için yeniden değerlendirilir ve probability ratio

$$\rho_t(\theta) = \exp\left(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)\right) \tag{60}$$

biçiminde hesaplanmaktadır.

Bu ratio, rollout sırasında seçilmiş bir action'ın probability'sinin policy update sonrasında ne kadar değiştiğini göstermektedir.

Eğer

$$\rho_t(\theta) = 1 \tag{61}$$

ise old policy ve new policy, ilgili action'a aynı probability'yi vermektedir.

Eğer

$$\rho_t(\theta) > 1 \tag{62}$$

ise action'ın new policy altındaki probability'si artmıştır.

Buna karşılık

$$\rho_t(\theta) < 1 \tag{63}$$

ise action'ın probability'si azalmıştır.

10 PPO Clipped Surrogate Objective

Vanilla policy-gradient yaklaşımında policy parameters'ın büyük bir gradient step ile güncellenmesi, new policy'nin old policy'den aşırı uzaklaşmasına neden olabilir.

PPO bu problemi, rollout'u oluşturan old policy ile updated policy arasındaki değişimi sınırlandırarak azaltır.

PPO'nun unclipped objective'i

$$L^{\mathrm{PG}}(\theta) = \mathbb{E}_t\left[\rho_t(\theta)\tilde{A}_t\right] \tag{64}$$

şeklindedir.

Ancak yalnızca bu objective kullanıldığında policy update aşırı büyük olabilir. Bu nedenle PPO clipped surrogate objective kullanır:

$$\begin{aligned} L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min \Big(&\rho_t(\theta)\tilde{A}_t, \\ &\mathrm{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)\tilde{A}_t\Big)\Big]. \end{aligned} \tag{65}$$

Baseline configuration için

$$\epsilon = 0.1 \tag{66}$$

olduğundan clipping interval

$$[1-\epsilon, 1+\epsilon] = [0.9, 1.1] \tag{67}$$

şeklindedir.

Eğer $\tilde{A}_t > 0$ ise selected action beklenenden daha iyi sonuç vermiştir. Bu durumda PPO action'ın probability'sini artırmaya çalışır. Ancak probability ratio'nun aşırı büyümesi durumunda

clipping, daha fazla policy değişiminin objective tarafından teşvik edilmesini sınırlar.

Eğer $\tilde{A}_t < 0$ ise selected action beklenenden daha kötü sonuç vermiştir. Bu durumda action'ın probability'si azaltılır; clipping yine aşırı büyük bir policy değişimini sınırlar.

Dolayısıyla PPO'nun temel fikri

$$\boxed{\text{Policy'yi geliştirirken new policy'yi old policy'ye yakın tutmak}} \tag{68}$$

şeklinde özetlenebilir.

11 Actor Optimization

Actor optimization'ın temel amacı, policy parameters θ'yı expected return'ü artıracak yönde güncellemektir.

Classical actor–critic yaklaşımında actor update doğrudan Policy Gradient prensibine dayanır. Tipik bir actor objective

$$J_{\mathrm{AC}}(\theta) = \mathbb{E}\left[\log \pi_\theta(a_t|s_t)\hat{A}_t\right] \tag{69}$$

şeklinde düşünülebilir.

Buna karşılık optimization algoritmaları genellikle bir loss function'ı minimize ettiği için classical actor–critic actor loss

$$L^{\mathrm{AC}}_{\mathrm{actor}} = -\mathbb{E}\left[\log \pi_\theta(a_t|s_t)\hat{A}_t\right] \tag{70}$$

şeklinde tanımlanabilir.

Dolayısıyla classical actor–critic için temel update yapısı

$$\theta \leftarrow \theta + \alpha_\pi \hat{A}_t \nabla_\theta \log \pi_\theta(a_t|s_t) \tag{71}$$

şeklinde ifade edilebilir.

Büyük gradient updateleri, policy'nin tek bir optimization step içerisinde önemli ölçüde değişmesine neden olabilir.

PPO, classical actor–critic'in bu policy-gradient yapısını korur; ancak actor update'in büyüklüğünü daha kontrollü hale getirmek amacıyla probability ratio ve clipping mechanism ekler.

PPO'da actor optimization için, önceki section'da tanımlanan $L^{\mathrm{CLIP}}(\theta)$ clipped surrogate objective kullanılmaktadır. Bu objective, old ve new policy arasındaki probability ratio'yu, normalize edilmiş advantage values'ı ve clipping mechanism'i birlikte dikkate alarak policy update'in büyüklüğünü sınırlandırmaktadır.

Dolayısıyla PPO, classical actor–critic'teki doğrudan advantage-weighted policy update yerine, policy değişimini old policy'ye göre kontrol eden clipped surrogate objective üzerinden actor parameters'ı optimize eder.

Optimization algoritmaları loss function minimize ettiği için, yalnızca clipped surrogate objective dikkate alındığında actor loss

$$L^{\mathrm{PPO}}_{\mathrm{actor}} = -L^{\mathrm{CLIP}}(\theta) \tag{72}$$

şeklinde ifade edilebilir.

Kullanılan implementation'da actor loss'a ayrıca policy entropy terimi eklenmektedir. Bu nedenle practical actor loss

$$L_{\text{actor}} = -L^{\text{CLIP}}(\theta) - c_H \mathcal{H}\left(\pi_\theta(\cdot|s_t)\right) \tag{73}$$

şeklinde ifade edilebilir.

Burada $\mathcal{H}$ policy entropy'sini, c_H ise entropy coefficient'i göstermektedir. Baseline configuration'da

$$c_H = 1.0753631441212628 \times 10^{-4} \tag{74}$$

kullanılmaktadır. Entropy terimi stochastic policy'nin exploration davranışını desteklemektedir.

Gaussian policy için entropy,

$$H(\pi_\theta) = \frac{1}{2}\log(2\pi e\sigma^2) = \frac{1}{2}\log(2\pi e) + \log\sigma$$

şeklinde ifade edilir. Bu nedenle entropy, Gaussian distribution'ın standard deviation parametresi σ ile doğrudan ilişkilidir. Standard deviation arttıkça distribution genişler ve farklı action'ların sample edilme olasılığı artar.

Actor loss minimize edildiğinden, negatif entropy terimi $-c_H H$, optimization sırasında entropy'nin artırılmasını teşvik eder. Kullanılan implementation'da $\log\sigma$ ayrı bir trainable parameter olduğundan,

$$\frac{\partial(-c_H H)}{\partial \log\sigma} = -c_H$$

elde edilir. Dolayısıyla entropy terimi, gradient-based optimization sırasında $\log\sigma$ değerini artırıcı yönde katkı sağlar.

Bu mekanizma, Gaussian policy'nin training sırasında çok erken daralmasını önlemeye yardımcı olarak exploration davranışını destekler. Bununla birlikte, standard deviation'ın gerçek güncellemesi PPO clipped objective ve entropy terimlerinin birleşik gradient'ine bağlıdır.

Bu practical actor loss'un minimize edilmesi,

$$L^{\text{CLIP}}(\theta) + c_H \mathcal{H}\left(\pi_\theta(\cdot|s_t)\right) \tag{75}$$

ifadesinin maximize edilmesine eşdeğerdir. Dolayısıyla actor parameters kavramsal olarak

$$\theta \leftarrow \theta + \alpha_\pi \nabla_\theta \left[L^{\text{CLIP}}(\theta) + c_H \mathcal{H}\left(\pi_\theta(\cdot|s_t)\right)\right] \tag{76}$$

şeklinde güncellenir.

Classical actor–critic ile PPO arasındaki temel fark bu noktada ortaya çıkar:

Classical Actor–Critic:	Advantage'a göre doğrudan policy update
PPO:	Advantage'a göre policy update
	+ old/new policy ratio
	+ clipping

(77)

Her iki yöntemde de Policy Gradient Theorem'da bulunan

$$\nabla_\theta \log \pi_\theta(a_t|s_t) \tag{78}$$

gibi gradient terimleri practical implementation içerisinde elle hesaplanmaz. Neural network computation graph üzerinden backpropagation ve automatic differentiation kullanılarak actor parameters'a ait gradient'ler otomatik olarak elde edilir.

Dolayısıyla backpropagation actor–critic yapısına genel bir optimization mekanizmasıdır; PPO'ya özel olan kısım, actor update öncesinde L^{CLIP} objective'inin kullanılmasıdır.

12 Critic Optimization

Critic'in amacı state-value estimate'i observed return'e yaklaştırmaktır.

Critic için training target

$$G_t \tag{79}$$

olarak alınır.

Value-function loss

$$L_V(\phi) = \frac{1}{2}\mathbb{E}_t\left[(V_\phi(s_t) - G_t)^2\right] \tag{80}$$

şeklindedir.

Critic parameters

$$\phi \leftarrow \phi - \alpha_V \nabla_\phi L_V(\phi) \tag{81}$$

ile güncellenir.

Training ilerledikçe critic'in

$$V_\phi(s) \approx V^\pi(s) \tag{82}$$

şeklinde daha doğru bir state-value approximation üretmesi beklenir.

13 Approximate KL-Divergence Based Update Constraint

PPO clipping mekanizmasına ek olarak, old policy ile new policy arasındaki değişim approximate Kullback–Leibler (KL) divergence kullanılarak izlenmektedir.

Approximate KL divergence

$$\widehat{D}_{\mathrm{KL}} \approx \mathbb{E}_t \left[\log \pi_{\theta_{\mathrm{old}}}(a_t|s_t) - \log \pi_\theta(a_t|s_t) \right] \tag{83}$$

şeklinde hesaplanır.

KL divergence'ın büyümesi, new policy'nin rollout'u oluşturan old policy'den belirgin şekilde uzaklaştığını gösterir. Kullanılan PPO implementation'ında actor update yalnızca

$$\widehat{D}_{\mathrm{KL}} \leq 1.5 D_{\mathrm{KL,target}} \tag{84}$$

koşulu sağlandığında uygulanmaktadır. Approximate KL değeri bu threshold'u aşarsa ilgili mini-batch için actor optimizer step uygulanmaz. Critic update ise bu KL constraint'ten bağımsız olarak devam etmektedir.

Böylece clipping mekanizmasına ek olarak aşırı büyük policy değişimlerini sınırlandıran ikinci bir mekanizma elde edilir.

14 Complete PPO Training Cycle

Bir PPO learning iteration, current policy kullanılarak environment'tan rollout data toplanmasıyla başlar. Her timestep'te current state s_t actor ve critic network'lerine uygulanır. Actor stochastic Gaussian policy üzerinden bir action sample eder:

$$a_t \sim \pi_\theta(a|s_t), \tag{85}$$

critic ise aynı state için

$$V_\phi(s_t) \tag{86}$$

value estimate'ini üretir.

Environment interaction sonucunda reward ve next state elde edilir:

$$(s_t, a_t) \rightarrow (r_{t+1}, s_{t+1}). \tag{87}$$

Rollout sırasında state, action, reward, critic value estimate ve selected action'ın current policy altındaki log-probability değeri buffer içerisinde saklanır. Bu log-probability daha sonraki optimization aşamasında old-policy probability olarak kullanılmaktadır.

Rollout tamamlandığında future discounted returns hesaplanır. Rollout gerçek terminal state ile bitmemişse final state'in critic value estimate'i kullanılarak future return bootstrap edilir. Time-limit truncation durumunda da terminal observation'ın critic value estimate'i hesaba katılmaktadır. Gerçek terminal state sonrasında ise future return sıfır kabul edilir.

Baseline configuration'da `use_gae=False` olduğundan advantage estimate

$$\hat{A}_t = G_t - V_\phi(s_t) \tag{88}$$

şeklinde hesaplanır.

Optimization öncesinde advantage values rollout batch üzerinde normalize edilir:

$$\tilde{A}_t = \frac{\hat{A}_t - \mu_{\hat{A}}}{\sigma_{\hat{A}} + 10^{-6}}. \tag{89}$$

Rollout sırasında saklanan old-policy log-probability ile current actor tarafından yeniden hesaplanan new-policy log-probability kullanılarak

$$\rho_t(\theta) = \exp\left(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)\right) \tag{90}$$

probability ratio elde edilir.

Bu ratio ve normalized advantage, önceki section'da tanımlanan $L^{\text{CLIP}}(\theta)$ PPO clipped surrogate objective içerisinde kullanılır.

Actor optimization sırasında clipped PPO objective'e ek olarak policy entropy regularization da kullanılmaktadır. Practical actor loss

$$L_{\text{actor}} = -L^{\text{CLIP}}(\theta) - c_H \mathcal{H}\left(\pi_\theta(\cdot|s_t)\right) \tag{91}$$

şeklindedir.

Actor optimizer step yalnızca

$$\widehat{D}_{\text{KL}} \leq 1.5 D_{\text{KL,target}} \tag{92}$$

KL constraint'i sağlandığında uygulanır.

Critic ise standard value-function loss

$$L_V(\phi) = \frac{1}{2}\mathbb{E}_t\left[\left(V_\phi(s_t) - G_t\right)^2\right] \tag{93}$$

minimize edilerek güncellenir. Baseline configuration'da `use_clipped_value=False` olduğundan value clipping uygulanmamaktadır. Critic update, actor için kullanılan KL constraint'ten bağımsız olarak gerçekleştirilir.

Aynı rollout data shuffled mini-batch'ler halinde birden fazla optimization epoch boyunca kullanılır. Optimization tamamlandıktan sonra updated actor policy bir sonraki rollout için current policy haline gelir ve yeni rollout updated policy ile toplanır.

Dolayısıyla PPO learning cycle kavramsal olarak

$$\boxed{\begin{aligned}&\text{Current Policy} \rightarrow \text{Rollout Collection}\\ &\qquad\qquad \rightarrow \text{Return / Advantage Estimation}\\ &\qquad\qquad \rightarrow \text{Advantage Normalization}\\ &\qquad\qquad \rightarrow \text{PPO Optimization}\\ &\qquad\qquad \rightarrow \text{Actor / Critic Update}\\ &\qquad\qquad \rightarrow \text{Updated Policy}\\ &\qquad\qquad \rightarrow \text{New Rollout}\end{aligned}} \tag{94}$$

şeklinde özetlenebilir.

15 Implementation-Level PPO Pseudo-Code

Bu bölümde PPO training procedure, kullanılan implementation'daki temel işlem sırasını gösterecek şekilde pseudo-code olarak özetlenmektedir.

Baseline configuration için

$$T = 150, \qquad N_{\text{rollout batch}} = 4, \tag{95}$$

dolayısıyla her rollout–optimization cycle içerisinde

$$N_{\text{rollout}} = 150 \times 4 = 600 \tag{96}$$

environment transition toplanmaktadır.

Optimization parameters

$$N_{\text{mini-batch}} = 128, \qquad K = 5 \tag{97}$$

olarak kullanılmaktadır. Toplam training budget ise

$$N_{\text{env,max}} = 300000 \tag{98}$$

environment step'tir.

```
Initialize actor parameters theta
Initialize critic parameters phi
Set total_steps = 0

while total_steps < 300000:

    Create a new rollout buffer

    for t = 1,...,150:

        For each of the 4 parallel environments:

            Observe current state s_t

            Actor:
                Construct Gaussian policy pi_theta(. | s_t)
                Sample action a_t
                Compute old log-probability logp_old

            Critic:
                Compute value estimate V_phi(s_t)
```

```
        Apply action a_t to environment

        Observe:
            reward r_(t+1)
            next state s_(t+1)
            terminal information

        Store in rollout buffer:
            s_t
            a_t
            r_(t+1)
            V_phi(s_t)
            logp_old
            terminal information

total_steps = total_steps + 600

Compute critic value of final rollout state

If rollout ends because of time-limit truncation:
    bootstrap using critic value of terminal observation

If rollout boundary is non-terminal:
    bootstrap using critic value of final rollout state

Compute discounted returns G_t backward through rollout

Since use_gae = False:
    A_hat_t = G_t - V_phi(s_t)

Normalize advantages:
    A_tilde =
        (A_hat - mean(A_hat))
        /
        (std(A_hat) + 1e-6)

for epoch = 1,...,5:

    Shuffle rollout samples

    Divide data into mini-batches of size 128
```

```
    for each mini-batch:

        Re-evaluate stored actions using current actor

        Compute new log-probability logp

        Compute probability ratio:
            rho = exp(logp - logp_old)

        Compute PPO clipped policy loss

        Compute policy entropy

        Compute approximate KL divergence

        if approximate_KL <= 1.5 * target_KL:

            Update actor using:
                clipped PPO loss
                + entropy regularization

        Compute critic value loss:
            L_V = 0.5 * (V_phi(s_t) - G_t)^2

        Update critic parameters phi

Use updated actor policy for next rollout
```

Rollout sampler her optimization epoch'unda samples'ı yeniden shuffle etmektedir. Kullanılan implementation'da incomplete final mini-batch kullanılmadığından, 600 rollout sample içerisinden her epoch'ta dört adet 128-sample mini-batch optimization için kullanılmaktadır.

Dolayısıyla bir optimization epoch içerisinde

$$4 \times 128 = 512 \tag{99}$$

sample actor ve critic optimization'a girer. Samples her epoch yeniden shuffle edildiğinden kullanılmayan samples her epoch aynı olmak zorunda değildir.

Toplam rollout–optimization cycle sayısı yaklaşık

$$\frac{300000}{600} = 500 \tag{100}$$

olarak elde edilir.

Buradaki pseudo-code PPO'nun learning mekanizmasına odaklanmaktadır. Evaluation, checkpoint saving ve logging gibi training infrastructure işlemleri bu akışın dışında bırakılmıştır.

16 Training and Deployment

Training phase ile deployment phase birbirinden ayrılmalıdır.

Training sırasında actor parameters θ ve critic parameters ϕ, environment'tan elde edilen rollout data kullanılarak sürekli olarak güncellenmektedir.

Training tamamlandıktan sonra actor policy freeze edilir:

$$\theta = \theta^*. \tag{101}$$

Deterministic evaluation sırasında stochastic sampling yerine Gaussian policy'nin mean action'ı kullanılır:

$$a_t = \mu_{\theta^*}(s_t). \tag{102}$$

Actor tarafından üretilen normalized action önce physical force command'a

$$u_{t,\mathrm{cmd}} = 10\mu_{\theta^*}(s_t) \tag{103}$$

şeklinde dönüştürülür.

Environment tarafından uygulanacak gerçek control input ise physical actuator limitleri dikkate alınarak

$$\boxed{u_t = \mathrm{clip}\left(10\mu_{\theta^*}(s_t), -10, 10\right)} \tag{104}$$

şeklinde elde edilir.

Deployment sırasında artık rollout collection, return calculation, advantage estimation, PPO clipping, critic optimization veya backpropagation yapılmaz. Yalnızca trained actor network current state'i kullanarak deterministic control action üretir.

Dolayısıyla training sonunda PPO tarafından öğrenilen controller, nonlinear bir state-feedback control law olarak

$$\boxed{u_t = \mathrm{clip}\left(10\mu_{\theta^*}(s_t), -10, 10\right)} \tag{105}$$

şeklinde ifade edilebilir.

Bu yapı, LQR'daki

$$u = -Kx \tag{106}$$

linear state-feedback law'a benzer şekilde yorumlanabilir. Ancak PPO durumunda state ile control action arasındaki mapping, trained actor neural network tarafından öğrenilmiş nonlinear bir function'dır.

17 Summary

Bu çalışmada kullanılan PPO controller, stochastic Gaussian actor ile learned state-value critic'i bir arada kullanarak CartPole stabilization policy'sini model-free ve on-policy şekilde öğrenmek-

tedir. Current policy ile rollout data toplanmakta, discounted returns ve normalized advantage estimates hesaplanmakta ve actor, old-policy ile current-policy arasındaki probability ratio'yu kullanan clipped PPO objective, entropy regularization ve KL-based update constraint altında optimize edilmektedir. Critic ise observed return'leri target olarak kullanan value-function loss üzerinden eğitilmektedir. Training tamamlandıktan sonra critic ve PPO optimization mekanizması kullanılmaz; trained actor'ın deterministic mean output'u bounded nonlinear state-feedback controller olarak uygulanır.